| Result | Size | Time | Cycles | GPU | SM Frequency | Process | Attributes |
|---|---|---|---|---|---|---|---|
| Current 575 - gemm_wmma_kernel | (8, 8, 1)x(128, 4, 1) | 70.24 us | 113,368 | 0 - NVIDIA GeForce RTX 4060 Laptop GPU | 1.61 Ghz | [19682] gemm_warp_spec_double | |

Summary
Details
Source
Context
Comments
Raw
Session
Compare
Tools
View
Export

## GPU Speed Of Light Throughput

Roofline Tensor Core

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

| | | | |
|---|---|---|---|
| Compute (SM) Throughput [%] | 37.83 | Duration [us] | 70.24 |
| Memory Throughput [%] | 27.39 | Elapsed Cycles [cycle] | 113,368 |
| L1/TEX Cache Throughput [%] | 28.83 | SM Active Cycles [cycle] | 107,456.50 |
| L2 Cache Throughput [%] | 13.10 | SM Frequency [Ghz] | 1.61 |
| DRAM Throughput [%] | 8.85 | DRAM Frequency [Ghz] | 7.98 |

**Small Grid** This kernel grid is too small to fill the available resources on this device, resulting in only 0.9 full waves across all SMs. Look at ▸Launch Statistics for more details.

▸ Key Performance Indicators

**Roofline Analysis** The ratio of peak float (FP32) to double (FP64) performance on this device is 64:1. The workload achieved close to 0% of this device's FP32 peak performance and 0% of its FP64 peak performance. See the Profiling Guide for more details on roofline analysis.

### Tensor Core Operations Roofline

Performance [OP/s] (1 = 1E+13)
10
1
0.1
1
10
100
1,000
10,000
100,000
Arithmetic Intensity [OP/byte]

| | # Operations | # Operations / Cycle | # Operations / s | Peak % | Peak Operations / Cycle | Peak Operations / s |
|---|---|---|---|---|---|---|
| Src:bf16 Dst:fp32 Sparsity:off | 0 | 0 | 0 | 0 | 12,288 | 19,781,569,020,501.14 |
| Src:bf16 Dst:fp32 Sparsity:on | 0 | 0 | 0 | 0 | 24,576 | 39,563,138,041,002.27 |
| Src:fp16 Dst:fp16 Sparsity:off | 0 | 0 | 0 | 0 | 24,576 | 39,563,138,041,002.27 |
| Src:fp16 Dst:fp16 Sparsity:on | 0 | 0 | 0 | 0 | 49,152 | 79,126,276,082,004.55 |
| Src:fp16 Dst:fp32 Sparsity:off | 268,435,456 | 2,373.97 | 3,821,689,293,849.66 | 19.32 | 12,288 | 19,781,569,020,501.14 |
| Src:fp16 Dst:fp32 Sparsity:on | 0 | 0 | 0 | 0 | 24,576 | 39,563,138,041,002.27 |
| Src:fp64 | 0 | 0 | 0 | 0 | 86.54 | 139,306,824,088.04 |
| Src:fp8 Sparsity:off | 0 | 0 | 0 | 0 | 49,152 | 79,126,276,082,004.55 |
| Src:fp8 Sparsity:on | 0 | 0 | 0 | 0 | 98,304 | 158,252,552,164,009... |
| Src:int1 | 0 | 0 | 0 | 0 | 393,216 | 633,010,208,656,036... |
| Src:int4 Sparsity:off | 0 | 0 | 0 | 0 | 98,304 | 158,252,552,164,009... |
| Src:int4 Sparsity:on | 0 | 0 | 0 | 0 | 196,608 | 316,505,104,328,018... |
| Src:int8 Sparsity:off | 0 | 0 | 0 | 0 | 49,152 | 79,126,276,082,004.55 |
| Src:int8 Sparsity:on | 0 | 0 | 0 | 0 | 98,304 | 158,252,552,164,009... |
| Src:tf32 Dst:fp32 Sparsity:off | 0 | 0 | 0 | 0 | 6,144 | 9,890,784,510,250.57 |
| Src:tf32 Dst:fp32 Sparsity:on | 0 | 0 | 0 | 0 | 12,288 | 19,781,569,020,501.14 |

## PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

| | | | |
|---|---|---|---|
| Maximum Sampling Interval [us] | 1 | # Pass Groups | 2 |
| Maximum Buffer Size [Mbytes] | 4 | - | - |

0s
ms
+0.02ms
+0.04ms
+0.06ms
+0.08ms
+0.1ms
+0.12ms
+0.14ms
+0.16ms
Overview
Average Active Warps Per Cycle
42.7 warp
0
Total Active Warps Per Cycle
1.02k warp
0
Blocks Launched
64 block
0
SM Active Cycles
1.64k cycle
0
Executed Ipc Active
2.75 inst/cycle
0
SM
SM ALU Pipe Throughput
100 %
0
SM FMA Light Pipe Throughput
100 %
0
SM FMA Heavy Pipe Throughput
100 %
0
SM Tensor Pipe Throughput
100 %
0
SM XU Pipe Throughput
100 %
0
DRAM
DRAM Throughput
100 %
0
DRAM Read Bandwidth
100 %
0
DRAM Write Bandwidth
100 %
0
L2 Cache
L2 Throughput
100 %
0
L2 Hit Rate
100 %
0
L1 Cache
L1 Throughput
100 %
0
L1 Writeback Throughput
483 cycle
0
L1 Hit Rate
100 %
0
L1 Wavefronts (Data)
1k
0
Workload Execution
gemm_wmma_kernel

## Compute Workload Analysis

Pipe Utilization (Elapsed Cycles)

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

| | | | |
|---|---|---|---|
| Executed Ipc Elapsed [inst/cycle] | 1.51 | SM Busy [%] | 37.83 |
| Executed Ipc Active [inst/cycle] | 1.59 | Issue Slots Busy [%] | 37.83 |
| Issued Ipc Active [inst/cycle] | 1.59 | | |

**Balanced** ALU is the highest-utilized pipeline (29.6%) based on elapsed cycles in the workload, taking into account the rates of its different instructions. It executes integer and logic operations. It is well-utilized, but should not be a bottleneck.

### Pipe Utilization (% of elapsed cycles)

0.0
25.0
50.0
75.0
100.0
ALU
FMA
Tensor (All)
Tensor (FP)
FP64
Tensor (INT)
Utilization [%]

### Pipe Utilization (% of peak instructions executed over elapsed cycles)

0.0
25.0
50.0
75.0
100.0
ALU
LSU
FMA
Tensor (FP)
ADU
CBU
Uniform
XU
FMA (FP16)
FP64 (DMMA)
FP64 (FP64)
TEX
Tensor (INT)
Utilization [%]

## Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

| | | | |
|---|---|---|---|
| Memory Throughput [Gbyte/s] | 22.60 | Mem Busy [%] | 26.24 |
| L1/TEX Hit Rate [%] | 34.00 | Max Bandwidth [%] | 27.39 |
| L2 Hit Rate [%] | 78.42 | Mem Pipes Busy [%] | 27.39 |
| L2 Compression Input Sectors [sector] | 0 | Local Memory Spilling Requests | 0 |
| L2 Compression Ratio | 0 | Local Memory Spilling Request Overhead [%] | 0 |
| L2 Compression Success Rate [%] | 0 | L2 Persisting Size [Mbyte] | 6.29 |

**L1TEX Global Load Access Pattern**
**Est. Speedup: 1.52%**
The memory access pattern for global loads from L1TEX might not be optimal. On average, only 30.2 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the ▸Source Counters section for uncoalesced global loads.

▸ Key Performance Indicators

**L1TEX Global Store Access Pattern**
**Est. Speedup: 13.70%**
The memory access pattern for global stores to L1TEX might not be optimal. On average, only 16.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the ▸Source Counters section for uncoalesced global stores.

▸ Key Performance Indicators

### Memory Chart

Values: Transfer Size  Inactivity: Greyed Out

Kernel
139.26 K Inst
Global
135.17 K Req
4.10 K Req
0.00 Inst
Local
0.00 Req
0.00 Req
0.00 Inst
Texture
0.00 Req
0.00 Inst
Surface
0.00 Req
0.00 Req
0.00 Inst
Load Global Store Shared
0.00 B
196.61 K Inst
Shared
65.54 K Req
131.07 K Req
L1/TEX Cache Hit Rate: 34.00 %
0.00 B
Shared Memory
159.69 K Req
ICC Hit Rate: 99.40 %
630.53 K Req
IMC Hit Rate: 99.64 %
1.02 K Req
IDC Hit Rate: 89.75 %
6.69 MB
694.56 KB
L2 Cache Hit Rate: 78.42 %
0.00 B
0.00 B
L2 Compression Ratio: 0.00
44.54 KB
1.15 K Req
408.00 Req
GCC Hit Rate: 87.69 %
0.00 B
0.00 B
System Memory
1.59 MB
0.00 B
Device Memory
0.00 B
0.00 B
Peer Memory
% Peak
100%
80%
60%
40%
20%
0%

## Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

| | | | |
|---|---|---|---|
| Active Warps Per Scheduler [warp] | 10.25 | No Eligible [%] | 59.64 |
| Eligible Warps Per Scheduler [warp] | 1.05 | One or More Eligible [%] | 40.36 |
| Issued Warp Per Scheduler | 0.40 | | |

**Issue Slot Utilization**
**Est. Local Speedup: 59.64%**
Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 2.5 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this workload allocates an average of 10.25 active warps per scheduler, but only an average of 1.05 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the ▸Warp State Statistics and ▸Source Counters sections can help, too.

▸ Key Performance Indicators

### Warps Per Scheduler

GPU Maximum Warps Per Scheduler
Theoretical Warps Per Scheduler
Active Warps Per Scheduler
Eligible Warps Per Scheduler
Issued Warp Per Scheduler
0.0
4.0
8.0
12.0
16.0

## Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

| | | | |
|---|---|---|---|
| Warp Cycles Per Issued Instruction [cycle] | 25.39 | Avg. Active Threads Per Warp | 32 |
| Warp Cycles Per Executed Instruction [cycle] | 25.44 | Avg. Not Predicated Off Threads Per Warp | 29.56 |

**Barrier Stalls**
**Est. Speedup: 47.51%**
On average, each warp of this workload spends 12.1 cycles being stalled waiting for sibling warps at a CTA barrier. A high number of warps waiting at a barrier is commonly caused by diverging code paths before a barrier. This causes some warps to wait a long time until other warps reach the synchronization point. Whenever possible, try to divide up the work into blocks of uniform workloads. If the block size is 512 threads or greater, consider splitting it into smaller groups. This can increase eligible warps without affecting occupancy, unless shared memory becomes a new occupancy limiter. Also, try to identify which barrier instruction causes the most stalls, and optimize the code executed before that synchronization point first. This stall type represents about 47.5% of the total average of 25.4 cycles between issuing two instructions.

▸ Key Performance Indicators

**Warp Stall** Check the ▸Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Profiling Guide provides more details on each stall reason.

### Warp State (All Cycles)

0.0
5.0
10.0
15.0
Warp States
Stall Barrier
Stall Long Scoreboard
Stall Math Pipe Throttle
Stall Wait
Stall Not Selected
Selected
Stall Short Scoreboard
Stall Dispatch Stall
Stall Branch Resolving
Stall IMC Miss
Stall No Instruction
Stall Drain
Stall MIO Throttle
Stall LG Throttle
Stall Misc
Stall Membar
Stall Sleeping
Stall Tex Throttle
Cycles per Instruction

## Instruction Statistics

Opcode Category Chart

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enabling parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

| | | | |
|---|---|---|---|
| Executed Instructions [inst] | 4,097,920 | Avg. Executed Instructions Per Scheduler [inst] | 42,686.67 |
| Issued Instructions [inst] | 4,106,032 | Avg. Issued Instructions Per Scheduler [inst] | 42,771.17 |

**FP32 Non-Fused Instructions**
**Est. Speedup: 3.77%**
This kernel executes 8192 fused and 8192 non-fused FP32 instructions. By converting pairs of non-fused instructions to their fused, higher-throughput equivalent, the achieved FP32 performance could be increased by up to 25% (relative to its current performance).

▸ Key Performance Indicators

### Executed Instruction Categories

0.0
2,000,000.0
4,000,000.0
Opcode Categories
Integer
Control
Load/Store
Movement
Floating Point
Miscellaneous
Uniform Datapath
Predicate
Conversion
Executed Warp-Level Instructions/Opcode

## NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

### NVLink Topology

The system does not have any NVLink connections.

## NVLink Tables

Detailed tables with properties for each NVLink.

### Logical NVLink Properties

The system does not have any NVLink connections.

## NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

### NUMA ID Table

| GPU ID | GPU Name | NUMA ID by CPU Affinity | CPU Affinity | NUMA ID by Memory Affinity |
|---|---|---|---|---|
| 0 | NVIDIA GeForce RTX 4060 Laptop GPU | 0 | 0-15 | 0 |

## Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

| | | | |
|---|---|---|---|
| Grid Size | 64 | Function Cache Configuration | CachePreferNone |
| Registers Per Thread [register/thread] | 37 | Static Shared Memory Per Block [Kbyte/block] | 10.75 |
| Block Size | 512 | Dynamic Shared Memory Per Block [byte/block] | 0 |
| Threads [thread] | 32,768 | Driver Shared Memory Per Block [Kbyte/block] | 1.02 |
| Waves Per SM | 0.89 | Shared Memory Configuration Size [Kbyte] | 65.54 |
| Uses Green Context | 0 | Stack Size | 1,024 |
| # SMs [SM] | 24 | # TPCs | 12 |
| Enabled TPC IDs | all | - | - |

## Occupancy

% Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

| | | | |
|---|---|---|---|
| Theoretical Occupancy [%] | 100 | Block Limit Registers [block] | 3 |
| Theoretical Active Warps per SM [warp] | 48 | Block Limit Shared Mem [block] | 5 |
| Achieved Occupancy [%] | 83.50 | Block Limit Warps [block] | 3 |
| Achieved Active Warps Per SM [warp] | 40.08 | Block Limit SM [block] | 24 |

**Achieved Occupancy**
**Est. Speedup: 16.50%**
The difference between calculated theoretical (100.0%) and measured achieved occupancy (83.5%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Best Practices Guide for more details on optimizing occupancy.

▸ Key Performance Indicators

### Impact of Varying Register Count Per Thread

% Occupancy
125
100
75
50
25
0
0
8
16
24
32
40
48
56
64
72
80
88
96
104
112
120
128
136
144
152
160
168
176
184
192
200
208
216
224
232
240
248
256
Registers Per Thread

### Impact of Varying Block Size

% Occupancy
125
100
75
50
25
0
0
32
64
96
128
160
192
224
256
288
320
352
384
416
448
480
512
544
576
608
640
672
704
736
768
800
832
864
896
928
960
992
1,024
Block Size

### Impact of Varying Shared Memory Usage Per Block

% Occupancy
125
100
75
50
25
0
0
3,184
6,368
9,552
12,736
15,920
19,104
22,288
25,472
28,656
31,840
35,024
38,208
41,392
44,576
47,760
50,944
54,128
57,312
60,496
63,680
66,864
70,048
73,232
76,416
79,600
82,784
85,968
89,152
92,336
95,520
98,704
101,888
Shared Memory Per Block

## GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

| | | | |
|---|---|---|---|
| Average SM Active Cycles [cycle] | 107,456.50 | Average L1 Active Cycles [cycle] | 107,456.50 |
| Average L2 Active Cycles [cycle] | 71,336.25 | Average SMSP Active Cycles [cycle] | 105,969.34 |
| Average DRAM Active Cycles [cycle] | 49,604 | Total SM Elapsed Cycles [cycle] | 2,713,784 |
| Total L1 Elapsed Cycles [cycle] | 2,713,784 | Total L2 Elapsed Cycles [cycle] | 1,795,536 |
| Total SMSP Elapsed Cycles [cycle] | 10,855,136 | Total DRAM Elapsed Cycles [cycle] | 2,242,560 |

### Workload Distribution

| | Average | Min | Max | Sum |
|---|---|---|---|---|
| SM Active Cycles | 107,456.50 | 103,443 | 111,076 | 2,578,956 |
| SMSP Active Cycles | 105,969.34 | 100,891 | 109,388 | 10,173,057 |
| L1 Active Cycles | 107,456.50 | 103,443 | 111,076 | 2,578,956 |
| L2 Active Cycles | 71,336.25 | 69,737 | 72,874 | 1,141,380 |
| DRAM Active Cycles | 49,604 | 49,536 | 49,664 | 198,416 |

## Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

| | | | |
|---|---|---|---|
| Branch Instructions [inst] | 353,152 | Branch Efficiency [%] | 100 |
| Branch Instructions Ratio [%] | 0.09 | Avg. Divergent Branches [branches] | 0 |

**Uncoalesced Global Accesses**
**Est. Speedup: 6.36%**
This kernel has uncoalesced global accesses resulting in a total of 32768 excessive sectors (10% of the total 327680 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

▸ Key Performance Indicators

### L2 Theoretical Sectors Global Excessive

| Location | Value | Value (%) |
|---|---|---|
| 0x768e032738d0 in gemm_wmma_kernel | 2,560 | 8 |
| 0x768e032738c0 in gemm_wmma_kernel | 2,560 | 8 |
| 0x768e032738b0 in gemm_wmma_kernel | 2,560 | 8 |
| 0x768e03273890 in gemm_wmma_kernel | 2,560 | 8 |
| 0x768e032736a0 in gemm_wmma_kernel | 2,560 | 8 |

### Warp Stall Sampling (All Samples)

| Location | Value | Value (%) |
|---|---|---|
| 0x768e032729d0 in gemm_wmma_ke... | 576 | 23 |
| 0x768e03272c40 in gemm_wmma_ke... | 564 | 23 |
| 0x768e03274d00 in gemm_wmma_ke... | 92 | 4 |
| 0x768e03273b30 in gemm_wmma_ke... | 91 | 4 |
| 0x768e032751b0 in gemm_wmma_ke... | 82 | 3 |

### Most Instructions Executed

| Location | Value | Value (%) |
|---|---|---|
| 0x768e032751e0 in gemm_wmma_ke... | 12,288 | 0 |
| 0x768e032751d0 in gemm_wmma_ke... | 12,288 | 0 |
| 0x768e032751c0 in gemm_wmma_ke... | 12,288 | 0 |
| 0x768e03275040 in gemm_wmma_ke... | 12,288 | 0 |
| 0x768e03275030 in gemm_wmma_ke... | 12,288 | 0 |

Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable individual sections to focus on selected performance aspects and make profiling faster.